

ABP 1DAM / 1DAW

Requisitos técnicos del módulo 0485 – Programación

API REST con persistencia en base de datos

Alcance: este documento define únicamente la parte evaluable desde Programación. La parte de Sistemas, BBDD, Entornos de Desarrollo, Lenguaje de Marcas y otros módulos se concretará en documentos propios y coordinados con este.

1. Finalidad dentro del ABP

El módulo 0485 – Programación evaluará que el alumnado sea capaz de construir una API REST funcional, organizada y conectada a una base de datos. No se evaluará aquí toda la infraestructura del proyecto global, sino la parte de lógica de programación, acceso a datos desde código, estructura del proyecto y exposición de servicios REST.

2. Producto que debe entregar el alumnado

Cada grupo deberá entregar una API REST relacionada con el dominio del ABP global. La API deberá permitir consultar, crear, modificar y eliminar datos de una base de datos elegida y documentada por el grupo.

Tecnología	Uso permitido	Observación
Java Spring Boot + JDBC	Válido	Recomendado si se sigue la práctica de tienda: Controller, DAO, JdbcTemplate, RowMapper y DTO.
Java Spring Boot + JPA-Hibernate	Válido	Se valorará como opción de mayor complejidad si se usan entidades, repositorios y relaciones.
Python Flask	Válido	Debe estar estructurado por capas. SQLAlchemy es opcional, pero se valorará positivamente.
Otros	¿?	Se debe consultar con los docentes de la asignatura para que acepten o no la propuesta.

3. Requisitos mínimos de Programación

- Implementar una API REST con respuestas en JSON.
- Conectarse a una base de datos real del Proyecto.
- Implementar un CRUD completo sobre al menos un recurso principal.
- Separar el código en capas o paquetes con responsabilidades claras.
- Utilizar DTOs para entrada y/o salida de datos, evitando exponer innecesariamente el modelo completo, si conviene.
- Devolver códigos HTTP adecuados y controlar errores básicos.
- Documentar la API.
- Incluir un README.md o MANUAL.md con instrucciones de ejecución y prueba.

4. Arquitectura esperada

La práctica de tienda puede tomarse como referencia de mínimos para entender el patrón DAO, el uso de DTOs, los endpoints y el manual técnico. En el ABP, para dar un paso más, se pide añadir una capa de servicio entre el controlador y el acceso a datos.

Capa	Responsabilidad
Controller / Routes	Recibe peticiones HTTP, valida datos básicos y devuelve respuestas.
Service	Contiene la lógica de negocio y coordina las operaciones.
DAO / Repository	Centraliza el acceso a la base de datos. No debe estar mezclado con el controlador.
Model / Entity	Representa los datos del dominio o las tablas de la base de datos.
DTO	Define qué datos entran y salen por la API.
Config	Centraliza configuración de aplicación y conexión a la base de datos.

Flujo recomendado: Cliente HTTP → Controller/Routes → Service → DAO/Repository → Model/Entity → Base de datos.

5. Endpoints mínimos

Cada grupo adaptará los nombres de las rutas al dominio de su proyecto. Antes de programar, deberá entregar una tabla similar a la siguiente.

Operación	Método	Ruta ejemplo	Código esperado
Listar registros	GET	/api/recurso	200 OK
Consultar por ID	GET	/api/recurso/{id}	200 OK / 404 Not Found
Crear registro	POST	/api/recurso	201 Created / 400 Bad Request
Modificar registro	PUT	/api/recurso/{id}	200 OK / 404 Not Found
Eliminar registro	DELETE	/api/recurso/{id}	204 No Content o 200 OK / 404 Not Found

6. Documentación de la API

La documentación de endpoints será breve, pero obligatoria. Deberá indicar, para cada endpoint: método HTTP, ruta, parámetros, JSON de entrada si lo hay, JSON de respuesta, códigos HTTP y posibles errores.

Campo	Ejemplo
Endpoint	GET /api/products/{id}
Descripción	Devuelve un producto por su identificador.
Parámetros	id: entero obligatorio en la URL.
Respuesta 200	{ "id": 1, "name": "Keyboard", "price": 35.5 }
Respuesta 404	{ "error": "Product not found" }

7. Entregables de Programación

- Repositorio Git con el código fuente.
- Script SQL, migración o instrucciones para crear la base de datos y datos de prueba.
- README.md o MANUAL.md con instalación, ejecución, configuración, endpoints y ejemplos JSON.

- API documentada con Swagger o similar
- Evidencias de prueba: colección Postman, capturas o ejemplos de llamadas.
- Breve explicación de la arquitectura y reparto de tareas del grupo.

9. Rúbrica orientativa de Programación

Criterio	Peso
Funcionamiento correcto de la API REST	20%
Persistencia real y acceso correcto a la base de datos	20%
Arquitectura por capas y separación de responsabilidades	20%
Uso correcto de DAO/Repository, JDBC/JPA/SQLAlchemy y DTOs	15%
Gestión de errores, validaciones básicas y códigos HTTP	10%
Documentación técnica: README/MANUAL, endpoints, JSON y ejecución	10%
Calidad del código, formato, Git y organización del proyecto	5%

10. Niveles de complejidad

Nivel	Descripción
Básico	CRUD funcional contra tablas no relacionadas entre sí.
Medio	Relaciones 1:1, 1: n entre tablas
Avanzado	Relaciones n:n entre tablas , validaciones más completas, JPA-Hibernate o SQLAlchemy bien aplicado. Uso de DTOs

11. Observaciones finales

- El dominio de los datos y el diseño completo de la base de datos deberán coordinarse con el módulo de BBDD.
- La configuración de despliegue, servidores, contenedores o red se tratará en la parte de Sistemas si corresponde.
- El frontend, formularios o presentación visual se coordinarán con Lenguaje de Marcas y los módulos implicados.
- No se aceptará una entrega con todo el código concentrado en un único archivo o clase si eso impide evaluar la arquitectura.
- El código deberá estar correctamente formateado, indentado y ser legible.
- No deben subirse contraseñas reales ni datos sensibles al repositorio
- Podéis usar IA en el Desarrollo, siempre y cuando seáis capaces de defender el código que os sugiera (se hará validación in situ)

Referencia de aula utilizada

Podéis usar como referencia las prácticas realizadas en las asignaturas de programación, lenguaje de marcas y APIs. En este ABP se mantiene esa base, pero se pide una estructura algo más completa y compleja, que se integre con el resto de módulos (BBDD, Sistemas, Entornos, ...).